Gatekeeper Deck

Gatekeeper

An RL environment that cannot be solved by allow-all, deny-all, or memorizing examples

Model-agnostic access-policy repair deterministic hidden verifier one-command Docker run

The task

The agent receives:

- a company world: users, nested groups, resources, tags, actions;
- a broken access policy;
- natural-language access requirements;
- 5-9 visible examples;
- a tool interface.

The agent must repair the policy and submit it.

The verifier scores the result over every user x action x resource x context variant.

Why access-policy repair?

- Real, high-stakes security work: authorization mistakes are costly.
- Exact semantics: default deny, deny-overrides-allow, conditions, exceptions, groups, wildcards.
- Naturally multi-step: inspect, hypothesize, edit, simulate, verify.
- Easy to create semantic mutations with measurable impact.
- Hard to fake: the full matrix exposes overfitting immediately.

The hidden verifier

```
score = max(0, 1 - (3 * false_allows + false_denies) / E_start)
```

- False allow is weighted 3x false deny.
- E_start is the broken policy's weighted error count.
- Full Cartesian matrix, not visible examples.
- Oracle actions exist only in the verifier.
- Shadow sample after each model turn for progress logs.

What the agent sees

Sanitized bundle only:

```
world
broken_policy
```

```
requirements
visible_examples
task_id, tier, seed
state_hash
```

Never exposed to the sandbox:

```
correct_policy
mutations
reference_decisions
matrix
e_start
oracle_actions
```

A test walks the bundle and checks every forbidden key.

Deny-list construction blocks trivial policies

Every correct policy contains:

1. protected allow-all s001;
2. protected deny-all-actions on tag:scope:restricted s002;
3. feature denies with conditions, groups and exceptions;
4. random denies.

Result:

```
| Policy | Score |
| Oracle | 1.000 |
| Broken / no-op | 0.000 |
| Allow-all | 0.000 |
| Deny-all | 0.000 |
```

The gate enforces all four on every task.

Six semantic mutations

- drop_statement
- widen_wildcard
- remove_condition
- remove_deny
- change_group
- flip_effect

The generator measures mutation impact on a hidden sample, rejects weak edits, tries dedicated feature targets first, and keeps protected baselines immutable.

Three services, two internal networks

```
model API
   ^
harness  (internet + tools + trusted)
 |   \
sandbox  verifier
tools    trusted
```

- sandbox: tools only, no internet, no verifier route.
- verifier: hidden answer key, no internet, no sandbox route.
- harness: trusted model loop, gate, budgets, logs.

Three images. docker compose up --build --abort-on-container-exit --exit-code-from harness.

Tools and limits

10 tools:

get_requirements, list_policies, lookup, simulate, put_policy, delete_policy, undo, diff, check_examples, submit.

Limits:

- 40 model steps
- 600s wall clock
- 10 tool calls per reply
- 4,000-char tool output
- 60s tool timeout
- 144K-char context budget

Every violation is logged and bounded.

Robust model interface

The parser accepts:

- native OpenAI tool_calls
- legacy function_call
- JSON objects/arrays in text, with or without fences
- tool/args and name/arguments shapes

Malformed JSON is a recoverable tool error, not a crash.
A fake OpenAI-compatible server exercises native, legacy, text, malformed, 25-call and no-tool scenarios in tests.

Gate before the model

```
| Check | Requirement |
| Oracle | >= 0.95 |
| No-op | <= 0.05 |
| Allow-all | <= 0.001 |
| Deny-all | <= 0.001 |
```

A gate failure stops the run with exit code 2.
The model is never called on a broken environment.

Determinism and evidence

- Seeded world, policy, mutations, examples and matrix.
- Canonical state hash.

- Verifier cache keyed by (task_id, seed).
- 12 tasks: easy, medium, hard.
- All 12 passed the four-baseline gate at seed 0.
- 77 local tests.
- baseline-results/gate.json and sample trajectories included.
- Real gpt-6-luna success on med_01 and easy_01; controlled failure logged.

Current limitations

- Docker Compose build and one-command run executed locally on 2026-10-25.
- Real gpt-6-luna calibration: med_01 and easy_01 solved in 5 steps;
  controlled no-op append failed at 0.0.
- Hard task generation is slower; standalone seeds took roughly 8-20s.
- mock/solver is a deterministic oracle-backed demo, not a model baseline.

Next: broader model sweep across seeds and a short demo video.

Why Gatekeeper is hard to break

- Hidden answer key and separate verifier network.
- Full-matrix scoring, not example matching.
- Deny-list shape kills allow-all and deny-all.
- Semantic mutations require understanding, not pattern matching.
- Strong gate blocks a trivially passable environment.
- Deterministic replay makes every result auditable.

Gatekeeper: repair the policy, not the benchmark.